



Versión del Documento 3.0

Equipo 4

- *Carpinteiro Tapia Mariam Ximena*
- *Espinosa Roque Rebeca*
- *García Díaz Michelle Ariadna*
- *Guillén Gilabert Lara*
- *Nava Córdova Mariana*
- *Sánchez Cortés Wendy*

Versión del Documento: 3.0

Fecha de actualización: 08 de mayo de 2026

Historial de cambios:

Versión	Cambios
1.0	Versión inicial
2.0	Integración frontend, autenticación, perfil de usuario
3.0	Nuevos CU: registrar animal, publicar, buscar, manifestar interés

Backend: Versión 3.0.0 (Tag publicado en el repositorio remoto).

- 5 nuevos casos de uso CU6, CU7, CU8, CU9, CU10

Frontend: Versión 2.0.0 (Tag publicado en el repositorio remoto).

- Búsqueda de animales con filtros (especie, tamaño, código postal, raza)
- Feed de publicaciones con carrusel de fotos
- Botón "Me interesa" en tarjetas de animales
- Formulario de publicación de animal
- Formulario de registro de animal

1. Introducción

1.1 Propósito del documento

1.2 Alcance del Sistema

1.3 Definiciones relevantes

2. Actores del Sistema

3. Requerimientos Funcionales

4. Casos de uso

CU1 - Registro de nuevo usuario

CU2 — Iniciar sesión

CU4 — Cerrar sesión

CU5 — Actualizar datos

CU6 — Registrar animal en adopción

CU7 — Publicar animal en adopción

CU8 — Buscar de animales publicados para adopción

CU9 — Manifestar interés en adopción

CU10 — Editar publicación

CU11 — Ver ficha informativa de un animal

5. Requerimientos No Funcionales

6. Tecnologías Utilizadas

6.1 Lenguaje de programación

6.2 Framework.

6.3 Base de datos

6.4 Herramientas relevantes utilizadas en el desarrollo.

6.5 Tecnologías del Frontend

7. Arquitectura general

Modelo MVC

Arquitectura REST

MVC adaptado a REST

Arquitectura en Capas del Sistema

Integración Frontend — Backend

8. Flujo de Autenticación

9. Modelo Conceptual de Datos

Entidad Usuario

Atributos

Consideraciones de seguridad

10. Limitaciones actuales

1. Introducción

1.1 Propósito del documento

Este documento tiene como propósito describir el estado actual del sistema desarrollado en la Iteración 2 del proyecto. Su objetivo es proporcionar una visión clara y estructurada de las funcionalidades implementadas, así como de los requerimientos funcionales y no funcionales que actualmente cumple el sistema.

Este documento funciona como una referencia para facilitar la comprensión de la organización del sistema, y como un medio de comunicación para stakeholders, permitiendo entender qué hace el sistema en su estado actual. Asimismo, sirve como base para futuras iteraciones, en las cuales se extenderán las funcionalidades del sistema de acuerdo con los objetivos del proyecto.

1.2 Alcance del Sistema

En la Iteración 2, el sistema implementa únicamente funcionalidades relacionadas con la gestión de usuarios dentro de la plataforma. Estas funcionalidades constituyen la base del sistema y permiten establecer un mecanismo de autenticación y manejo de sesiones.

Actualmente, el sistema permite:

- El registro de usuarios mediante la captura de información básica.
- La autenticación de usuarios a través de credenciales (correo electrónico y contraseña).
- El cierre de sesión, invalidando la sesión activa del usuario.
- La consulta de la información del usuario autenticado mediante un endpoint protegido.
- Actualizar el username, el nombre, el apellido y el código postal.

En esta etapa, el sistema cuenta con un frontend funcional conectado al backend, implementando un flujo completo desde la interfaz de usuario hasta la persistencia en base de datos.

En la Iteración 3 se incorporan funcionalidades de gestión de animales: registro de animales, publicación en feed, búsqueda con filtros, y manifestación de interés con notificación por correo al dueño.

El sistema prioriza desde esta iteración aspectos básicos de seguridad, como el almacenamiento cifrado de contraseñas y la validación de credenciales.

1.3 Definiciones relevantes

- **Usuario registrado:** Persona que ha creado una cuenta proporcionando su nombre, correo electrónico y código postal, y que tiene acceso a las funcionalidades de publicación y contacto.
- **Adopción responsable:** Proceso facilitado por el sistema mediante el cual una persona asume el compromiso ético y legal de brindar cuidado, protección y bienestar a un animal.
- **Animal en adopción:** Perro o gato registrado en la plataforma por un usuario, disponible para ser adoptado.
- **Ubicación aproximada:** Referencia geográfica basada únicamente en el *código postal*, utilizada para mostrar al animal en un mapa sin revelar la dirección exacta del domicilio.
- **Ficha informativa del animal:** Sección informativa dentro del sistema que muestra los datos detallados del animal, incluyendo nombre, descripción, tipo, raza, fotografías y ubicación aproximada.
- **Interés de adopción:** Funcionalidad que permite a un usuario manifestar interés en adoptar un animal, generando una notificación automática por correo electrónico al usuario que lo registro.
- **Autenticación:** Proceso mediante el cual el sistema verifica la identidad de un usuario mediante credenciales.
- **Contraseña cifrada:** Contraseña almacenada mediante un algoritmo de cifrado o hash de manera que no pueda leerse en texto plano.
- **Dato de contacto sensible:** Información personal que el sistema protege y no muestra directamente en la interfaz pública para evitar spam o riesgos de seguridad.

- **API externa:** Servicio externo que permite obtener información adicional.

2. Actores del Sistema

- **Usuario registrado**

Persona que tiene una cuenta en el sistema y puede acceder a funcionalidades protegidas como consulta de su información y cierre de sesión.

- **Usuario visitante**

Persona que no cuenta con una sesión activa en el sistema. Puede registrarse o iniciar sesión.

- **Soporte técnico**

Personal encargado del mantenimiento del sistema. No interactúa directamente con las funcionalidades implementadas en esta iteración.

3. Requerimientos Funcionales

RF1: Registro de usuarios

El sistema permite el registro de usuarios capturando el nombre completo, una dirección de correo electrónico válida, el código postal de residencia y una contraseña para el acceso a la cuenta.

RF2: Autenticación de usuarios

El sistema autentifica a los usuarios registrados mediante su correo electrónico y contraseña antes de permitir el acceso a las funcionalidades de la plataforma.

RF3: Consulta de usuario autenticado

El sistema permite obtener la información de usuario autenticado mediante un endpoint protegido (`/me`).

RF4: Cierre de sesión

El sistema permite al usuario registrado cerrar su sesión activa, invalidando el token de autenticación.

RF5: Actualización de datos del perfil de usuario

El sistema permite al usuario autenticado actualizar su username, nombre, apellido y código postal.

RF6: Registro de animal en adopción

El sistema permite registrar perros o gatos con nombre, especie, tamaño, descripción, código postal y fotos.

RF7: Publicación de animal

El sistema permite crear publicaciones visibles en el feed de otros usuarios.

RF8: Búsqueda de animales

El sistema permite filtrar animales por especie, tamaño, código postal y raza.

RF9: Manifestar interés

El sistema permite a un usuario marcar interés en un animal, notificando por correo al dueño con los datos del interesado.

4. Casos de uso

flowchart TB

actorV((Usuario Visitante))
actorR((Usuario Registrado))

subgraph Sistema

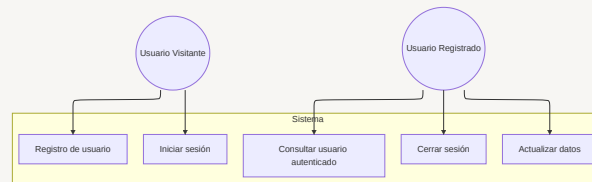
UC1[Registro de usuario]
UC2[Iniciar sesión]
UC3[Consultar usuario autenticado (/me)]

```

UC4[Cerrar sesión]
UC5[Actualizar datos]
UC6[Registrar animal en adopción]
end

actorV --> UC1
actorV --> UC2
actorR --> UC3
actorR --> UC4
actorR --> UC5
actorR --> UC6

```



CU1 - Registro de nuevo usuario

Actores:	Usuario visitante (no registrado)
Descripción breve:	Permite a una persona crear una cuenta en el sistema para poder acceder a las funcionalidades de la plataforma.

Precondición: El usuario no tiene una sesión activa en la plataforma.

Flujo principal:

1. El usuario accede a la opción "Crear cuenta".
2. El usuario ingresa su nombre completo, correo electrónico y código postal.

3. El usuario establece una contraseña para su cuenta.
4. El sistema valida que el correo tenga un formato valido.
5. El sistema cifra la contraseña antes de guardarla.
6. El Sistema confirma la creación de la cuenta y redirige al inicio de sesión.

Flujos alternativos:

A1: Si el correo ya existe en la base datos, el sistema muestra un error y solicita uno diferente.

A2: Si el formato de correo es inválido, el sistema muestra un error y solicita un correo valido.

Poscondición: La cuenta del usuario queda habilitada en el sistema.

CU2 — Iniciar sesión

Actores	Usuario registrado, Servicio de autenticación
Descripción	El usuario accede al sistema mediante sus credenciales para utilizar las funcionalidades de la plataforma.

Precondición: El usuario tiene una cuenta registrada y no tiene sesión activa.

Flujo principal:

1. El usuario accede a la opción "Iniciar sesión".
2. El usuario ingresa su correo electrónico y contraseña.
3. El sistema envía las credenciales al servicio de autenticación.
4. El servicio valida las credenciales y emite un token de sesión.
5. El el sistema devuelve una respuesta con el token de autenticación.

Flujos alternativos:

A1: Si las credenciales son incorrectas, el sistema muestra un mensaje de error sin

especificar cuál campo es incorrecto.

A2: Si el usuario falla 5 intentos consecutivos, el sistema bloquea temporalmente el acceso.

Postcondición: El usuario accede a la plataforma con una sesión autenticada activa.

CU4 — Cerrar sesión

Actores	Usuario registrado, Servicio de autenticación
Descripción	El usuario finaliza su sesión activa en la plataforma para proteger su cuenta y evitar accesos no autorizados en el dispositivo actual.

Precondición: El usuario debe tener una sesión autenticada activa en el sistema.

Flujo principal:

1. El usuario selecciona la opción "Cerrar sesión".
2. El sistema solicita al Servicio de autenticación la invalidación del token de sesión actual.
3. El Servicio de autenticación confirma la revocación del token o su expiración inmediata.
4. El sistema elimina las cookies o datos de sesión almacenados localmente en el navegador/aplicación.
5. El sistema redirige al usuario a la página de inicio o al formulario de Inicio de Sesión.

Flujos alternativos:

- **A1: Error de conexión con el servicio de identidad:** Si el servicio externo no responde, el sistema debe forzar de todas formas el borrado de la sesión local para impedir que el usuario siga navegando, aunque el token no se haya invalidado en el servidor inmediatamente.
- **A2: Cierre por inactividad:** Si el usuario no interactúa con el sistema por un tiempo prolongado, el sistema ejecuta el cierre de sesión de forma

automática.

Postcondición: El token de sesión queda invalidado en la base de datos y el usuario vuelve al estado de "Usuario no autenticado", perdiendo acceso a las funcionalidades privadas.

CU5 — Actualizar datos

Actores	Usuario registrado
Descripción	Permite al usuario autenticado actualizar sus datos personales (nombre, apellido, username y código postal.)

Precondición: El usuario debe tener una sesión autenticada activa en el sistema.

Flujo principal:

1. El usuario accede a la vista de "Actualización de perfil".
2. El sistema muestra un formulario con los datos actuales del usuario .
3. El usuario modifica uno o más campos y guarda los cambios.
4. El sistema envía la solicitud HTTP al backend mediante el método PUT al endpoint /users, incluyendo el token de autenticación.
5. El backend válida la validez del token y procesa la actualización en la base de datos.
6. El sistema confirma la actualización exitosa y actualiza la información de la vista.

Flujos alternativos:

- **A1: Token inválido o ausente.**

CU6 — Registrar animal en adopción

Actores:	Usuario registrado
Descripción breve:	El usuario da de alta un perro o gato en la plataforma. Este animal podrá ser publicado posteriormente.

Precondición: El usuario ha iniciado sesión exitosamente.

Flujo principal:

1. El usuario selecciona la opción "Registrar animal en adopción"
2. El usuario ingresa el nombre, tipo de animal y código postal de la ubicación. Opcionalmente, fecha de nacimiento, raza, descripción y tamaño.
3. El usuario adjunta al menos una fotografía del animal.
4. El sistema valida que todos los campos obligatorios estén completos.
5. El sistema guarda la información.

Flujos alternativos:

A1: Si falta información obligatoria, el sistema muestra mensaje de error.

Poscondición: El animal en adopción queda registrado y listo para ser publicado en un post.

CU7 — Publicar animal en adopción

Actores:	Usuario registrado y un animal en adopción
Descripción breve:	El usuario realiza una publicación en la plataforma, esta es compartida con los otros usuarios.

Precondición: El usuario ha iniciado sesión exitosamente. El usuario ya ha registrado anteriormente al animal cuya publicación desea hacer.

Flujo principal:

1. El usuario selecciona la opción "Hacer publicación de animal en adopción"

2. El usuario selecciona uno de los animales que ha registrado
3. El usuario escribe una descripción.
4. El sistema valida que todos los campos obligatorios estén completos.
5. El sistema guarda la información y lo muestra a otros usuarios en el feed de la homepage.

Flujos alternativos:

A1: Si falta información obligatoria, el sistema muestra mensaje de error.

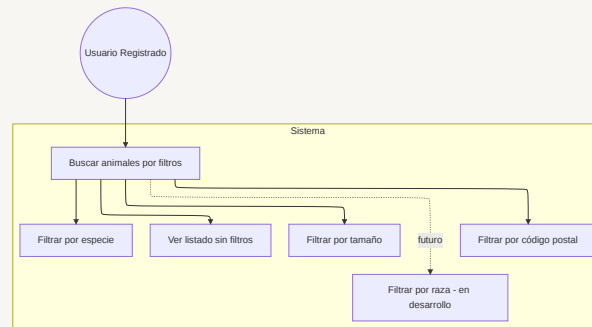
A2: Si se trata de publicar un animal que pertenece a otro usuario, el sistema impide la publicación.

Poscondición: El post queda publicado y visible para otros usuarios en la plataforma.

CU8 — Buscar de animales publicados para adopción

```
flowchart TB
    actorR((Usuario Registrado))
    subgraph Sistema
        UC1[Buscar animales por filtros]
        UC2[Ver listado sin filtros]
        UC3[Filtrar por especie]
        UC4[Filtrar por tamaño]
        UC5[Filtrar por código postal]
        UC6[Filtrar por raza - en desarrollo]
    end
    actorR --> UC1
    UC1 --> UC2
    UC1 --> UC3
```

UC1 --> UC4
 UC1 --> UC5
 UC1 -->|futuro| UC6



Actores:	Usuario registrado (autenticado)
Descripción breve:	Permite a un usuario autenticado consultar el listado de animales disponibles para adopción, con la posibilidad de aplicar filtros opcionales para refinar los resultados.

Precondición: El usuario tiene una sesión activa en la plataforma (token Bearer válido).

Flujo principal:

1. El usuario accede a la sección de animales en adopción.
2. El sistema realiza una petición `GET /animals` con el token de autenticación.
3. El sistema retorna todos los animales registrados, ordenados del más reciente al más antiguo.
4. El usuario visualiza el listado con la información de cada animal (especie, tamaño, código postal, raza y fotos).

Flujos alternativos:

A1 — Filtro por especie: El usuario selecciona una especie (`DOG` o `CAT`). El sistema retorna únicamente los animales que coincidan con dicha especie.

A2 — Filtro por tamaño: El usuario selecciona un tamaño (`small`, `medium`, `large` o `extra_large`). El sistema retorna únicamente los animales que coincidan con ese tamaño.

A3 — Filtro por código postal: El usuario ingresa un código postal (5–10 caracteres). El sistema retorna únicamente los animales cuyo código postal coincida de forma exacta.

A4 — Combinación de filtros: El usuario aplica dos o más filtros simultáneamente. El sistema aplica todos los criterios de forma conjunta y retorna solo los animales que cumplan todas las condiciones.

A5 — Sin resultados: Si ningún animal coincide con los filtros aplicados, el sistema retorna un arreglo vacío `[]` con estado `200 OK`. No se genera un error `404`.

Flujos en desarrollo:

D1 — Filtro por raza (`breedName`): El usuario escribe total o parcialmente el nombre de una raza. El sistema realiza una búsqueda insensible a mayúsculas/minúsculas. *Este filtro está pendiente de integración con la API externa de razas; su comportamiento puede ser incompleto en esta versión.*

Restricciones (fuera de alcance en esta versión):

- No es posible seleccionar múltiples valores para un mismo filtro (ej. `DOG` y `CAT` simultáneamente).
- No existe búsqueda por nombre o descripción libre del animal.
- No existe filtro por edad o rango de fechas.
- No existe búsqueda por proximidad geográfica; el código postal es una coincidencia exacta.

Poscondición: El usuario visualiza el listado de animales disponibles, filtrado según los criterios seleccionados.

CU9 — Manifestar interés en adopción

Actores	Usuario registrado
Descripción	El usuario expresa interés en adoptar un animal publicado en la plataforma. El sistema registra el interés y notifica por correo electrónico al dueño del animal con los datos del interesado.

Precondición: El usuario tiene una sesión activa en la plataforma.

Flujo principal:

1. El usuario visualiza la ficha de un animal en la sección de búsqueda.
2. El usuario presiona el botón "Me interesa".
3. El sistema registra el interés en la base de datos asociando al usuario con el animal.
4. El sistema envía un correo electrónico al dueño del animal con el nombre y correo del interesado, así como la ficha del animal.
5. El botón cambia a "¡Interés enviado! 🐾" y se deshabilita para evitar duplicados.

Flujos alternativos:

A1: Si el usuario ya había manifestado interés en ese animal previamente, el sistema rechaza la solicitud y el interés no se registra de nuevo.

Poscondición: El interés queda registrado en la base de datos y el dueño del animal recibe una notificación por correo electrónico.

CU10 — Editar publicación

Actores:	Usuario registrado y un animal en adopción
Descripción breve:	El usuario modifica la descripción o el estado de una publicación de animal que él mismo registró.

Precondición: El usuario ha iniciado sesión exitosamente y es el dueño de la publicación a editar.

Flujo principal:

1. El usuario selecciona la publicación que desea editar.
2. El sistema muestra la información actual de la publicación.
3. El usuario modifica la descripción, el estado, o ambos campos.
4. El usuario confirma los cambios.
5. El sistema valida que el usuario sea el propietario del animal vinculado a la publicación.
6. El sistema guarda los cambios y muestra la publicación actualizada.

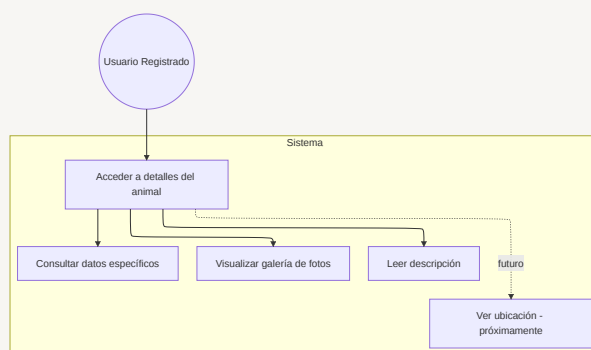
Flujos alternativos:

A1: Si el usuario no es el propietario del animal vinculado, el sistema rechaza la operación y muestra un mensaje de error.

A2: Si no se modifica ningún campo, el sistema mantiene la información existente sin cambios.

Poscondición: La publicación queda actualizada con la nueva información proporcionada por el usuario.

CU11 — Ver ficha informativa de un animal



Actores:	Usuario registrado con autenticación
Descripción breve:	El usuario puede visualizar una versión completa de los detalles sobre una mascota específica.

Precondición: El usuario tiene una sesión activa (token de sesión válido) y tiene una mascota en mente de la que quiere saber más información ubicada en el `PostFeed` o encontrada mediante `AnimalSearch`.

Flujo principal:

1. El usuario hace clic en el botón "More info" de la tarjeta de un animal.
2. El sistema redirige a la ruta `/animal/:id` y realiza una petición `GET /animals/{id}` con el token de autenticación.
3. El sistema devuelve el objeto con los detalles del animal.
4. El sistema renderiza la ficha informativa.

5. El usuario visualiza el nombre, raza, especie, la galería de fotos, tamaño, descripción, etc.

Flujos alternativos:

- **A1 — Navegación de fotos:** El usuario interactúa con los controles del carrusel de fotos para visualizar las distintas fotos del animal.
- **A2 — Regreso al listado:** El usuario presiona el botón "Volver al listado" o utiliza la navegación del navegador. El sistema lo retorna a la vista previa sin perder la información que se mostraba antes de entrar a la ficha.
- **A3 — Animal no encontrado:** Si el `id` en la URL no existe o fue eliminado, el sistema captura el error 404 a través del `ExceptionHandler` y muestra el mensaje "Animal no encontrado".

Flujos en desarrollo:

- **D1 — Visualización de ubicación (Mapa):** Se guardó un espacio para poder mostrar un mapa interactivo que muestre la ubicación del animal basado en el código postal registrado.

Restricciones (fuera de alcance en esta versión):

- No es posible realizar acciones de contacto o adopción directamente desde esta ficha.
- La información es de solo lectura.

Poscondición: El usuario queda informado sobre las características del animal para decidir si le interesa adoptar o no al animal.

5. Requerimientos No Funcionales

RNF1 - Seguridad

Las contraseñas se almacenan mediante un algoritmo de cifrado seguro.

RNF2 – Rendimiento

Las solicitudes del sistema deben de responder en un tiempo promedio no mayor

a 3 segundos bajo carga normal.

RNF3 - Disponibilidad

El sistema está disponible para los usuarios al menos el 99% del tiempo durante cada mes de operación.

RNF4 - Usabilidad

El sistema permite a un usuario registrarse e iniciar sesión de manera intuitiva sin necesidad de capacitación previa.

RNF6 - Privacidad

El sistema garantiza la protección de los datos personales de los usuarios, evitando la exposición pública de información sensible como direcciones exactas, contraseñas y correos electrónicos dentro de la interfaz de la plataforma. La información del usuario únicamente podrá ser utilizada para fines relacionados con la funcionalidad del sistema.

RNF6 - Compatibilidad

El sistema deberá ser compatible con los principales navegadores web modernos (Chrome, Firefox, Edge, etc) y ser accesible tanto desde dispositivos de escritorio como desde dispositivos móviles mediante diseño responsivo.

RNF7 - Escalabilidad

El sistema soporta múltiples usuarios concurrentes realizando operaciones de autenticación sin degradación perceptible del rendimiento.

RNF8 - Mantenibilidad

El código fuente del sistema se encuentra documentado y estructurado de forma modular, permitiendo que un desarrollador nuevo pueda identificar y corregir un error en un módulo sin afectar el funcionamiento del resto del sistema.

RNF11 - Integridad de Datos

El sistema garantiza la integridad de datos en la base de datos haciendo uso de políticas de mantenimiento adecuadas, no permitiendo la existencia de datos aislados o incompletos según la definición de la base de datos.

6. Tecnologías Utilizadas

6.1 Lenguaje de programación

| *Kotlin*

Kotlin es una opción acertada para el desarrollo de aplicaciones backend modernas por las grandes ventajas que ofrece en comparación a otros lenguajes del campo. Kotlin destaca por su sintaxis concisa y expresiva, lo que permite reducir significativamente la cantidad de código, facilitando así la lectura y el mantenimiento de código. Además, tiene un soporte para programación concurrente mediante *coroutines*, las cuales permiten escribir código asíncrono de manera más sencilla y legible, evitando la complejidad asociada con la gestión de múltiples hilos de ejecución. Otra de sus características importantes es su sistema de tipos con manejo explícito de valores nulos, este enfoque ayuda a prevenir errores asociados a referencias nulas.

6.2 Framework.

| *Spring Boot*

Spring Boot fue seleccionado como Framework por su compatibilidad con Java, y por ende, con Kotlin. Según IMB, Spring ofrece soporte integrado para tareas típicas que una aplicación necesita realizar, como enlace de datos, conversión de tipos, validación, manejo de excepciones, gestión de recursos y eventos, internacionalización y más. Spring permite la *inyección de dependencias* y

proporciona configuraciones automáticas. Además, tiene una amplia comunidad, documentación extensa y soporte para tecnologías modernas.

6.3 Base de datos

| PostgreSQL

PostgreSQL fue elegido por su estabilidad, alto rendimiento y cumplimiento de los estándares SQL. Sigue la filosofía de código abierto y tiene una una amplia comunidad. Tiene soporte para transacciones complejas, integridad de datos y optimización de consultas. Todo esto es deseable para gestionar la información de nuestros usuarios y sus animales a adopción de forma segura y eficiente.

6.4 Herramientas relevantes utilizadas en el desarrollo.

| Hibernate

Hibernate se utilizó como herramienta de ORM, permitiendo la interacción entes el sistema desarrollaron en Kotlin y la base de datos de PostgreSQL mediante objetos en lugar de consultar SQL directas. Esto reduce significativamente la cantidad de código para acceder a la base datos.

| Postman

Es nuestra principal herramienta para probar y depurar la API REST, ya que permite enviar solicitudes HTTP a los distintos endpoints del sistema y analizar la respuesta obtenida, lo cual facilitan la verificación del correcto funcionamiento de la API.

| Git

Git es sin duda el mejor sistema de control de versiones para gestionar el desarrollo proyectos en equipo. Podemos registra los cambios, mantener un

historial de las modificaciones y sobre todo propiciar el trabajo colaborativo entre los miembros de nuestro equipo mediante el uso del repositorio compartido.

(Las secciones 6.1 a 6.4 se mantienen igual.)

6.5 Tecnologías del Frontend

React: Biblioteca de JavaScript utilizada para construir la interfaz de usuario mediante componentes reutilizables (bloques independientes de código que representan partes visuales de la aplicación). Se eligió por su amplia adopción en la industria, su ecosistema maduro y la facilidad para gestionar el estado de la aplicación (los datos que cambian mientras el usuario interactúa con la pantalla).

Vite: Como su nombre lo indica, Vite es una herramienta de construcción (build tool) que acelera y hace más eficiente el desarrollo web, específicamente para levantar el servidor de desarrollo y compilar el proyecto. Como resultado nos ayuda a obtener aplicaciones más ligeras, tiempos de carga mínimos y un flujo de trabajo fluido.

React Router: Biblioteca utilizada para gestionar la navegación entre vistas dentro de la aplicación. Permite definir rutas (direcciones URL como /login /home /perfil) y controlar el acceso a páginas según si el usuario está autenticado o no.

Fetch API: Mecanismo nativo del navegador utilizado para realizar peticiones HTTP (solicitudes de datos entre el frontend y el backend) hacia el servidor. Se eligió por no requerir dependencias externas adicionales y ser suficiente para las necesidades actuales del sistema.

sessionStorage: Mecanismo de almacenamiento del navegador utilizado para guardar el token de sesión (una cadena de texto única que identifica al usuario autenticado) durante su visita. Se eligió por su naturaleza temporal: los datos se eliminan automáticamente al cerrar el navegador, lo que reduce riesgos de seguridad.

7. Arquitectura general

El sistema está compuesto por dos componentes independientes que se comunican entre sí a través de HTTP (el protocolo de comunicación de la web): el frontend y el backend.

El backend del sistema se implementa siguiendo el patrón **MVC adaptado a una arquitectura REST, organizado mediante una arquitectura en capas**. Este enfoque permite separar responsabilidades dentro del sistema, facilitando la mantenibilidad, la escalabilidad y la claridad en la estructura del código.

Modelo MVC

El patrón **MVC (Model–View–Controller)** es un modelo de diseño de software que separa una aplicación en tres componentes principales con el objetivo de organizar el código y facilitar su mantenimiento.

- **Model (Modelo)**: representa los datos del sistema y las reglas asociadas a estos. Se encarga de gestionar la información y su interacción con la base de datos.
- **View (Vista)**: es la capa encargada de presentar la información al usuario.
- **Controller (Controlador)**: actúa como intermediario entre el modelo y la vista, recibiendo las solicitudes del usuario, procesándolas y devolviendo una respuesta adecuada.

Arquitectura REST

La arquitectura **REST (Representational State Transfer)** es un estilo arquitectónico utilizado para diseñar servicios web que permiten la comunicación entre sistemas a través del protocolo HTTP.

En un sistema REST, los recursos del sistema se exponen mediante **endpoints** accesibles mediante operaciones HTTP como:

- **GET** para obtener información
- **POST** para crear nuevos recursos
- **PUT/PATCH** para actualizar información
- **DELETE** para eliminar recursos

Las respuestas del servidor se envían en formatos estructurados **JSON**.

MVC adaptado a REST

En el contexto de un *backend* REST, el patrón MVC se adapta de manera que:

- Los **Controllers** reciben las solicitudes HTTP de los clientes.
- El **Model** representa las entidades del sistema y su persistencia.
- La **View** es reemplazada por **respuestas JSON** enviadas al cliente.

Arquitectura en Capas del Sistema

Además del patrón MVC, el sistema se organiza mediante una **arquitectura en capas**, lo que permite dividir la aplicación en componentes con responsabilidades específicas. En la implementación actual se utilizan tres capas principales:

- **Controller:**

Esta capa es responsable de manejar las solicitudes HTTP provenientes de los clientes. En esta capa se definen los *endpoints* de la API REST que permiten interactuar con el sistema.

Los *controllers* reciben las peticiones del cliente, validan la estructura básica de los datos recibidos y delegan la lógica de negocio a la capa de servicios. Una vez que el procesamiento ha sido realizado, los *controllers* construyen y devuelven las respuestas HTTP correspondientes al cliente.

Esta capa actúa como el punto de entrada al sistema para todas las operaciones relacionadas con los usuarios.

- **Service:**

Esta capa contiene la lógica de negocio del sistema. Su responsabilidad principal es implementar las reglas y procesos que definen el comportamiento de la aplicación.

Los servicios reciben las solicitudes provenientes de los *controllers* y coordinan las operaciones necesarias para cumplirlas. Esto puede incluir validaciones adicionales, generación de tokens de sesión, procesamiento de datos y comunicación con la capa de persistencia.

Esta capa permite mantener separada la lógica de negocio del manejo de las solicitudes HTTP y del acceso directo a la base de datos.

- **Repository**

Esta capa se encarga de la comunicación con la base de datos. Su función es abstraer las operaciones de persistencia y proporcionar métodos para consultar, almacenar y actualizar la información del sistema.

Los *repositories* permiten acceder a las entidades almacenadas en la base de datos mediante consultas definidas a través del *framework* de persistencia utilizado. De esta manera, el resto del sistema puede interactuar con los datos sin depender directamente de los detalles de la base de datos.

Esta separación facilita la mantenibilidad del sistema y permite modificar o extender la lógica de persistencia sin afectar las demás capas de la aplicación.

En el flujo de lo implementado en nuestro sistema, **la arquitectura se ve aplicada** de la siguiente manera:

1. El cliente envía una solicitud HTTP con sus credenciales (email y contraseña).
2. El **Controller** recibe la solicitud y delega el procesamiento.
3. El **Service** valida las credenciales y gestiona la lógica de autenticación.
4. El **Repository** consulta la base de datos para verificar la información del usuario.
5. Si las credenciales son correctas, se genera un token y se guarda en la base de datos.
6. El sistema devuelve el token al cliente como respuesta.

Integración Frontend — Backend

A partir de la Iteración 2, el sistema cuenta con una interfaz de usuario que se comunica directamente con la API REST desarrollada en la Iteración 1. Ambos componentes operan de forma independiente y se conectan exclusivamente a través de peticiones HTTP.

El **frontend** es una aplicación web desarrollada en React. Su responsabilidad es presentar las vistas al usuario, capturar sus acciones y comunicarse con el backend para obtener o modificar información. No contiene lógica de negocio (reglas que definen cómo funciona el sistema): toda esa responsabilidad recae en el backend.

El **backend** es la API REST organizada en capas (Controller, Service, Repository) desarrollada en Spring Boot con Kotlin, tal como se describe en la sección anterior. Es el único componente que interactúa directamente con la base de datos.

La **comunicación** entre ambos componentes se realiza mediante peticiones HTTP estándar. El frontend consume los endpoints (puntos de acceso de la API) expuestos por el backend, enviando y recibiendo datos en formato JSON (un formato de texto estructurado para intercambiar información). Cuando una operación requiere autenticación, el frontend incluye el token de sesión en el encabezado Authorization de cada petición.

El flujo general del sistema integrado funciona de la siguiente manera:

1. El usuario interactúa con la interfaz visual en el navegador.
2. El frontend construye una petición HTTP y la envía al backend.
3. El backend procesa la solicitud a través de sus capas (Controller → Service → Repository).
4. El backend devuelve una respuesta en formato JSON.
5. El frontend interpreta la respuesta y actualiza la interfaz del usuario.

8. Flujo de Autenticación

El flujo de autenticación del sistema funciona de la siguiente manera:

Cuando el usuario desea iniciar sesión, ingresa sus credenciales (correo electrónico y contraseña) en el formulario de login del frontend. Estas credenciales son enviadas al backend, que las valida contra la base de datos y, si son correctas, responde con un token de sesión (una cadena de texto única generada por el servidor para identificar al usuario durante esa sesión).

Una vez recibido, el token se guarda en el sessionStorage del navegador (un espacio de almacenamiento temporal que solo existe mientras la pestaña del navegador está abierta). Este token actúa como una credencial de identidad: cada vez que el usuario realiza una acción que requiere autenticación, el token es enviado automáticamente junto con la petición al backend en el encabezado Authorization (campo del mensaje HTTP que transporta información de identidad).

Las vistas que requieren autenticación están protegidas mediante rutas privadas (componentes que verifican la existencia del token antes de mostrar el contenido). Si el usuario intenta acceder a ellas sin tener un token activo en su sesión, el sistema lo redirige automáticamente a la pantalla de login.

Actualmente las vistas protegidas son:

- */home*
- */profile*
- */profile/edit*

Si el token no existe o ha sido eliminado (por ejemplo, al cerrar sesión o cerrar el navegador), el sistema considera que el usuario no está autenticado y le impide el acceso a cualquier funcionalidad que lo requiera. Se redirige a la página */login*.

9. Modelo Conceptual de Datos

Entidad Usuario

La entidad describe a los usuarios que interactúan con el sistema: tienen la capacidad de publicar o ver publicaciones animales en adopción

Atributos

Los siguientes atributos son almacenados en la base de datos y son solicitados como parte del registro de usuario.

Los siguientes atributos conforman la entidad Usuario y son almacenados en la base de datos:

Atributo	Propósito
id_user	Identificador único del usuario. Generado automáticamente por la base de datos.
first_name	Nombre de pila del usuario. Requerido, máximo 20 caracteres.
last_name	Apellido(s) del usuario. Requerido, máximo 50 caracteres.
email	Correo electrónico del usuario. Se utiliza como identificador para el inicio de sesión.

Atributo	Propósito
user_password	Contraseña del usuario. Se almacena como hash, nunca en texto plano.
zipcode	Código postal del usuario. Representa su ubicación aproximada. Debe tener entre 5 y 10 caracteres.
birth_date	Fecha de nacimiento del usuario. No puede ser una fecha futura.
gender	Género del usuario.
register_date	Fecha y hora en que el usuario se registró. Generada automáticamente.
token	Token de sesión activa. Se genera al iniciar sesión y se elimina al cerrar sesión.

Consideraciones de seguridad

- La contraseña nunca se almacena en texto plano. Antes de guardarse, se le aplica el algoritmo de hash SHA-256, de forma que no pueda recuperarse su valor original.
- El token de sesión se genera como un UUID aleatorio al momento del inicio de sesión, lo que lo hace prácticamente imposible de predecir.
- La contraseña es excluida de las respuestas que el sistema envía al cliente, o en su defecto es reemplazada por asteriscos.
- Las credenciales de conexión a la base de datos se gestionan mediante variables de entorno, evitando que queden expuestas en el código fuente.

10. Limitaciones actuales

Las siguientes limitaciones han sido identificadas en el estado actual del sistema y serán resueltas en iteraciones futuras:

Campos birth_date y gender sin capturar en el registro

El formulario de registro no solicita fecha de nacimiento ni género al usuario. Al persistir el registro, estos campos se llenan con valores por defecto temporales.

Hash de contraseña sin salt

El algoritmo SHA-256 utilizado actualmente es vulnerable a ataques de diccionario (intentos masivos de contraseñas comunes). Se recomienda migrar a un algoritmo más seguro como bcrypt en iteraciones futuras.

Protección de rutas solo en el cliente

La protección de rutas se aplica únicamente en el frontend verificando si existe un token en sessionStorage (almacenamiento temporal del navegador). Un token manipulado manualmente podría intentar acceder a recursos del backend sin ser bloqueado de forma centralizada.

Sin persistencia de sesión entre pestañas

El token se almacena en sessionStorage, lo que significa que si el usuario abre una nueva pestaña del navegador deberá iniciar sesión nuevamente, ya que sessionStorage no se comparte entre pestañas.

Imposibilidad de actualizar la contraseña de usuario

Solo se permiten modificar los siguientes campos del perfil de un usuario: nombre, apellido, nombre de usuario y código postal. Todavía no existe forma de modificar la contraseña del perfil. Recomendamos implementar esta funcionalidad desde una página nueva y no desde *profile/edit* como el resto de campos.

Notificación por correo dependiente de configuración manual del servidor

El envío de correos al dueño del animal al manifestar interés requiere que el servidor tenga configuradas variables de entorno con una cuenta Gmail y su contraseña de aplicación. Si estas variables no están presentes, el interés se registra correctamente en la base de datos pero el correo no se envía, sin que el usuario reciba ningún aviso de fallo.

Sin validación de autointerés en adopción

El sistema no impide que un usuario marque "Me interesa" en un animal que él mismo registró. En ese caso, el correo de notificación se enviaría al mismo usuario que presionó el botón, lo cual carece de utilidad. Se recomienda agregar una validación en el backend que rechace la solicitud si el usuario autenticado es el mismo dueño del animal.

Estado del botón "Me interesa" sin persistencia entre sesiones

Al presionar el botón "Me interesa", este cambia visualmente a "¡Interés enviado!" únicamente durante la sesión activa en el navegador. Si el usuario recarga la página o cierra y vuelve a abrir el sitio, el botón aparecerá nuevamente disponible aunque el interés ya haya sido registrado. El backend rechazará correctamente el duplicado, pero la experiencia de usuario resulta inconsistente.